Long story short, here we can easily keep track of the *rank* (aka size) of each tree, using linear extra space and performing constant-time extra operations in method `merge`, where we will update rank only for trees' roots.

When we merge two trees, we will make sure to set as child the tree with the smallest number of nodes, as shown in figure 5.10. It can be can be proven by induction that this tree will also be the one with smallest height: this means the new tree will either have the same height as the old one, or just have its height increased by 1. Even more, it is provable that the height of a tree can't be increased more than a logarithmic number of times.

As logarithm grows really slowly (for instance `ln(1000) ~= 10`, `ln(1000000) ~= 20`) this is, in practice, already a good result, by and large sufficient for most applications.

However, if you are writing some really critical core code (like a kernel or firmware code) ... you might want to do even better.

Why? Well, because you can. And sometimes also because you need to: if you shave 0.001ms over an operation you will repeat a billion times, you already saved 16 minutes of computation.

Most of the time, in our job as developers, performance isn’t the only metric to decide about this kind of improvements: first, it depends if you are saving those 16 minutes over a computation taking an hour or a day (needless to say, in the latter situation the gain would be